



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

 Contact Me

Data science/Agent

[Agent] 6. Agent 시스템: Memory

endingo 2025. 5. 3. 19:57 ⋮

Memory

- AI Agent가 이전 상태나 대화기록을 유지하고 재사용할 수 있도록 하는 기능
- 유저와 AI간의 대화를 저장하고 지속적인 맥락을 유지
- 단기기억과 장기기억 정보를 제어하기 위한 로직을 넣을 수 있음.

메모리의 종류 2가지

- Short-term memory: MemorySaver => memory
- Long-term memory: SqliteSaver => file, db

MemorySaver

- LangGraph에서 제공하는 체크포인트 저장소
 - 그래프 실행 중 생성되는 state를 자동으로 저장하고 필요시 복원 할 수 있게
- 해주는 메모리 기반 저장소

1. 기억력 없는 chatbot



정훈이의 공부일지



```
messages: Annotated[List, add_messages]
```

2) Node 정의

chatbot 이라는 노드 정리

```
llm = chatOpenAI(model="")
```

```
def chatbot(state: State):
    return {"messages": [llm.invoke(state["messages"])]}
```

3) 그래프 정의

```
builder = StateGraph(State)
```

```
builder.add_node("chatbot", chatbot)
```

```
builder.add_edge(START, "chatbot")
```

```
builder.add_edge("chatbot", END)
```

```
graph = builder.compile()
```

```
Human Message
```

```
내 이름이 뭐라고?
```

```
===== Ai Message =====
```

```
죄송하지만, 당신의 이름을 알 수 있는 정보가 없습니다. 자신의 이름을 알려주
```

2. Agent에 메모리 추가

MemorySaver: Short-term Memory

SqliteSaver: Long-term Memory

1) MemorySaver

- Short-term Memory

- LangGraph에서 제공하는 체크포인트 저장소

- 그래프 실행 중 생성되는 상태[state]를 자동으로 저장하고 필요 시 복원할 수 있게 해주는 메모리 기반 저장소

```
from langgraph.checkpoint.memory import MemorySaver
memory = MemorySaver()
```

체크포인트와 함께 그래프 컴파일



정훈이의 공부일지



기억 시키려고 스레드를 지정하는 것이다!!
대화의 흐름을 구분하기 위한 스레드를 설정해준다.
해당 키에 상태 데이터를 저장합니다.

예시 1)

```
thread_1 = {"configurable": {"thread_id": "1"}}

messages = [HumanMessage(content="3과 4를 더하라")]
messages = agent_with_memory.invoke({"messages": messages}, thread_1)

for m in messages['messages']:
    m.pretty_print()
```

예시 2)

```
messages = [HumanMessage(content="거기에 2를 곱하라.")]
messages = agent_with_memory.invoke({"messages": messages}, thread_1)

for m in messages['messages']:
    m.pretty_print()
```

- 새로운 대화

```
thread_2 = {"configurable": {"thread_id": "2"}}

messages = [HumanMessage(content="금리에 대한 정의를 알려줘.")]
messages = agent_with_memory.invoke({"messages": messages}, thread_2)

for m in messages['messages']:
    m.pretty_print()
```

다시 thread_1을 실행하면 thread_1 내용을 불러와서 실행한다.

```
# 첫 번째 입력 메시지를 정의
messages = [HumanMessage(content="마지막으로 계산했던 숫자가 뭐였지?")]

# 그래프 실행 (MemorySaver가 활성화된 그래프 사용)
messages = agent_with_memory.invoke({"messages": messages}, thread_1)
# 결과 메시지를 출력
for m in messages['messages']:
    m.pretty_print()
```



정훈이의 공부일지



```

===== Ai Message =====
3과 4를 더하면 7입니다.
===== Human Message =====
거기에 2를 곱하라.
===== Ai Message =====
7에 2를 곱하면 14입니다.
===== Human Message =====
마지막으로 계산했던 숫자가 뭐였지?
===== Ai Message =====
마지막으로 계산했던 숫자는 14입니다.

```

3. 메모리 다루기

1) 메모리 조회

- thread1을 지정하면 thread_1에 모두 자동으로 저장된다

```
agent_with_memory.get_state(thread_1).values
```

thread1 values를 보면 그동안 기억했던 정보를 보여준다.

- 마지막 메세지만 보여준다.

```
temp = agent_with_memory.get_state(thread_1).values
```

```
temp['messages'][-1].content
```

2) 메모리 삭제

- 공식적으로 제공되지 않지만, thread_id를 새로 시작하거나 memory 인스턴스를 다시 지정

3) 최근 메시지 n개만 기억

- 기억할 메세지 개수 제한하기

```
def filter_messages(messages: list):
```

```
    return messages[-2:]
```

```
def chatbot(state: State):
```

```
    messages = filter_messages(state["messages"])
```



정훈이의 공부일지



4. 메시지 요약

1) State 선언

```
class State(MessagesState):
    summary: str
```

2) 노드와 분기함수

요약 메시지가 있으면 요약 메세지 리스트에 누적시킴

```
def call_model(state: State):
    summary = state.get("summary", "")
    if summary:
        # 요약을 system prompt로 넣어 대화 맥락 유지
        system_message = f"이전 대화 요약: {summary}"
        messages = [SystemMessage(content=system_message)] + state["messages"]
    else:
        messages = state["messages"]

    response = llm.invoke(messages)
    return {"messages": [response]}
```

만약 메시지 6개 초과하면 summarize_conversation 노드로 분기 그렇지 않으면 바로 END

```
def should_continue(state: State) -> Literal["summarize_conversation", END]:
    if len(state["messages"]) > 6:
        return "summarize_conversation"
    return END
```

1

[Data science](#) > [Agent](#) 카테고리의 다른 글

[Agent] 7. Agent 시스템: RAG (0)

[Agent] 4. Agent 시스템 State, Node, Edge 리... (0)

[Agent] 2. Agent 시스템 State, Node, Edge 리... (0)

[Agent] 5. Agent 시스템: Tools (0)

[Agent] 3. Agent 시스템 State, Node, Edge 리... (0)



정훈이의 공부일지



정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

댓글 0

endingo

내용을 입력하세요.

등록

공지사항

최근에 올라온 글

- [Agent] 7. Agent 시스템: RAG
- [Agent] 6. Agent 시스템: Me...
- [Langchain] 5. 메모리와 체인
- [Langchain] 4. Prompt와 Ou...

최근에 달린 댓글

- endingo님 솔직하고 담백한 표...
- 잘 읽고 갑니다
- 오늘도 화이팅!!!
- 덕분에 좋은 정보 얻어 갑니다 ! ...

Total

317

Today	0
Yesterday	0

링크

TAG

[more](#)

- github
- git

«		2025/05							»	
일	월	화	수	목	금	토				
					1	2	3			
4	5	6	7	8	9	10				
11	12	13	14	15	16	17				
18	19	20	21	22	23	24				
25	26	27	28	29	30	31				

글 보관함

- 2025/05 (8)
- 2025/04 (34)
- 2025/03 (7)

Blog is powered by Tistory / Designed by Tistory



이정훈 개발자

AI와 백엔드, 데이터 분석을 좋아하는 개발자입니다.
다양한 문제 해결과 창의적인 개발을 추구합니다.

이메일 보내기